CHALLENGE SHEET

Vaultcorp

Six-stage SaaS exploit chain from client-side secret leak to SSRF-driven RCE via an unauthenticated Hangfire dashboard (CVE-2021-41238).

DIFFICULTY	TRACK	HOST PORT	LAB ID
HARD	WEB	5000	vaultcorp

EASY | MEDIUM | HARD | INSANE

VERIFY THE SOLVE

```
• python3 ../scripts/check.py vaultcorp
```

Brief

A custom-built, multi-stage web exploitation challenge simulating a realistic SaaS application. Chains six real, independently-documented vulnerabilities into a single exploit path from anonymous visitor to Remote Code Execution.

Setup

```
cp .env.example .env   # then edit .env and set the real FLAG value
docker compose up --build
```

Open `http://localhost:5000`.

Goal

Chain all six stages to reach Remote Code Execution and recover the flag. Verify the solve:

```
python3 ../../../scripts/check.py .
```

What This Is

A custom-built, multi-stage web exploitation challenge simulating a realistic SaaS application built with ASP.NET Core. The challenge chains six real-world vulnerabilities — each independently documented in CVEs and bug bounty reports — into a single exploit path from anonymous visitor to Remote Code Execution.

Repo Layout

```
.
```

```
    │
    ├── docker-compose.yml          ← everything needed to run the challenge
    ├── .env.example                ← copy to .env and set the real FLAG (never committed
)
    │
    ├── docs/                        ← all written evidence/report + maintainer docs
    │   ├── README.md
    │   ├── player-briefing.mdx     ← player-facing description (no spoilers)
    │   ├── challenge-design.mdx
    │   ├── official-solution.mdx   ← full spoiler solve, including the narrated,
    │   │                               screenshot-illustrated walkthrough
    │   ├── operations.mdx
    │   ├── security-report.mdx
    │   ├── remediation.mdx
    │   ├── validation-report.mdx
    │   └── images/                 ← screenshots referenced from official-solution.mdx
    │
    └── src/                        ← full source code (private — never ship to
participants)
        ├── VaultCorp.csproj
        ├── Program.cs
        ├── appsettings.json
        ├── Dockerfile
        ├── Data/
        │   ├── AppDbContext.cs
        │   └── Models/
        │       ├── Tenant.cs
        │       └── User.cs
        ├── Services/
        │   ├── TenantService.cs
        │   └── UrlFetchService.cs
        ├── Jobs/
        │   ├── CommandExecutorJob.cs
        │   └── DatabaseCleanupJob.cs
        ├── Pages/
        │   ├── Shared/_Layout.cshtml
        │   ├── _ViewImports.cshtml
        │   ├── _ViewStart.cshtml
        │   ├── Index.cshtml
        │   ├── Register.cshtml
        │   ├── Login.cshtml
        │   ├── SupportLogin.cshtml
        │   ├── Dashboard.cshtml
        │   ├── Profile.cshtml
        │   ├── Logout.cshtml
        │   ├── Admin/
        │   │   ├── Index.cshtml
        │   │   └── Import.cshtml
        │   └── Assets/
        │       └── UserUtils.cshtml
        └── wwwroot/
            ├── css/site.css
            └── js/
                ├── site.js
                ├── user-utils.js      ← served dynamically (per-tenant)
                └── internal.js
```

Where to Start

If you want to…	Go to
Understand what the challenge tests	`docs/challenge-design.mdx` (”Skills exercised”)
See the player-facing description (no spoilers)	`docs/player-briefing.mdx`
Reproduce the full solve step by step, with screenshots	`docs/official-solution.mdx` (”Full narrated walkthrough”)
Understand the security root cause	`docs/security-report.mdx`
See mitigation recommendations	`docs/remediation.mdx`
Run the challenge	`docker-compose.yml`

Running the Challenge

```
cp .env.example .env   # then edit .env and set the real FLAG value
docker compose up --build
```

Then navigate to `http://localhost:5000`.

Design Summary

- **Per-player isolation** — each `Register` creates a fully isolated tenant with unique admin credentials. The flag itself is baked into the deployed image at build time and is the same for every tenant in a given deployment (see `docs/challenge-design.mdx`, ”Determinism and flag design”) — isolation applies to account data, not to the flag value. Players cannot interfere with each other’s tenant data.
- **Chain dependency** — no stage can be skipped. Stages 1→2→3→4→5→6 must be completed in order.
- **All real** — every vulnerability has a published CVE, OWASP entry, or documented bug bounty report. Nothing is contrived.
- **Single container** — no complex infrastructure. One `docker compose up` and the challenge is live.

Vulnerability Chain

```
Stage 1  JS credential leak (XOR+Base64 obfuscation)
Stage 2  Hidden support login endpoint (directory brute-force)
Stage 3  SSRF with WAF bypass via IP obfuscation (127.1, decimal, octal, hex)
Stage 4  Internal Hangfire dashboard access via loopback trust (CVE-2021-41238)
Stage 5  Hangfire job injection → Remote Code Execution
Stage 6  Flag read via static file serving
```